LAB 2 REPORT

Chetna Parmar | Containerization & Kubernetes

Name: Chetna Parmar
Lab 2: Containerization & Kubernetes Orchestration | CO1, CO2
Tools: Docker 24, Minikube, kubectl, Docker Hub, VS Code, Ubuntu 24.04
Sample App: Node.js Hello App (chetna-app) | Image: chetna/app:v1

Exp 1 (5 marks): Containerize Sample App using Docker

Aim: Write Dockerfile, build image, run container.

Dockerfile:

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Commands:

```
$ docker build -t chetna/app:v1 .
$ docker images
$ docker run -d -p 3000:3000 --name chetna-app chetna/app:v1
$ docker ps
$ curl localhost:3000 -> Hello from Chetna Container
```

Image Layers: Base alpine, WORKDIR, npm install, COPY, CMD - each layer cached.
Observation: Image size 145 MB, container starts in 2 sec.
Screenshot: docker build + docker ps + browser localhost:3000

Exp 2 (5 marks): Manage Docker Containers & Images

Commands:

```
$ docker ps -a
$ docker start chetna-app
$ docker stop chetna-app
$ docker rm chetna-app
$ docker rmi chetna/app:v1
$ docker tag chetna/app:v1 chetna/app:latest
$ docker push chetna/app:latest
$ docker pull chetna/app:latest
$ docker logs chetna-app
$ docker exec -it chetna-app sh
```

Docker Compose - docker-compose.yml:

```
version: '3'
services:
  web:
    build: .
    ports: ['3000:3000']
  db:
    image: mongo:6
    ports: ['27017:27017']
```

```
$ docker compose up -d
```

LAB 2 REPORT

Chetna Parmar | Docker Storage & Network

Exp 3 (5 marks): Docker Storage

1. Loss of container-layer data:

```
$ docker run -it --name temp1 alpine sh
$ echo hello > /data.txt
$ docker rm temp1
$ docker run -it alpine cat /data.txt -> File Not Found - Data Lost
```

2. Named Volume - Permanent:

```
$ docker volume create chetna-vol
$ docker run -d -v chetna-vol:/app/data --name app1 chetna/app:v1
$ docker run -d -v chetna-vol:/app/data --name app2 chetna/app:v1
$ docker volume inspect chetna-vol
```

Data survives even after rm app1, app2 sees same data.

3. Bind Mount:

```
$ mkdir ~/mydata
$ docker run -d -v ~/mydata:/app/data --name app3 chetna/app:v1
```

Host file change instantly visible in container.

Observation: Volumes = persistent, Bind mount = live dev sync.

Exp 4 (5 marks): Docker Network

Aim: Two containers communicate via network, name & IP.

```
$ docker network create chetna-net
$ docker run -d --name web1 --network chetna-net nginx
$ docker run -d --name web2 --network chetna-net nginx
$ docker inspect web1 | grep IPAddress -> 172.18.0.2
$ docker inspect web2 | grep IPAddress -> 172.18.0.3
```

Test by Name:

```
$ docker exec web1 ping -c 2 web2 -> Reply from web2
$ docker exec web1 curl http://web2
```

Test by IP:

```
$ docker exec web1 ping -c 2 172.18.0.3 -> Reply
```

```
$ docker network ls
$ docker network inspect chetna-net
```

Observation: Docker DNS resolves container names, IP also works.

Screenshot: ping by name + ping by IP both success.

LAB 2 REPORT

Chetna Parmar | Kubernetes Deploy & Scaling

Exp 5 (5 marks): Deploy App to Kubernetes (Minikube)

```
$ minikube start --driver=docker
$ kubectl cluster-info
$ kubectl get nodes
```

deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: chetna-deployment
spec:
  replicas: 2
  selector: {matchLabels: {app: chetna}}
  template: {metadata: {labels: {app: chetna}}}
  template: {spec: {containers: [{name: chetna, image: chetna/app:v1}]}}
```

service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: chetna-service
spec:
  type: NodePort
  selector: {app: chetna}
  ports: [{port: 80, targetPort: 3000}]
```

Commands:

```
$ kubectl apply -f deployment.yaml
$ kubectl apply -f service.yaml
$ kubectl get pods
$ kubectl get svc
$ minikube service chetna-service --url
$ kubectl get deployments
```

Result: 2 Pods Running, Service NodePort accessible.

Screenshot: kubectl get pods + get svc

Exp 6 (5 marks): K8s Deployment Strategies, Rollback & Scaling

Rolling Update:

```
$ kubectl set image deployment/chetna-deployment chetna=chetna/app:v2
$ kubectl rollout status deployment/chetna-deployment
$ kubectl rollout history deployment/chetna-deployment
```

Rollback:

```
$ kubectl rollout undo deployment/chetna-deployment
$ kubectl rollout history deployment/chetna-deployment -> v1 restored
```

Scaling:

```
$ kubectl scale deployment chetna-deployment --replicas=4
$ kubectl get pods -> 4 Pods Running
$ kubectl scale deployment chetna-deployment --replicas=2
$ kubectl autoscale deployment chetna-deployment --min=2 --max=5 --cpu-percent=70
```

Observation: Zero downtime in rolling update, rollback in 10 sec,
scaling instant

LAB 2 REPORT

Chetna Parmar | CD & Final Report

Exp 7 (5 marks): Continuous Deployment using Kubernetes + CI/CD

Jenkins Pipeline - Jenkinsfile:

pipeline {

```
  agent any
  stages {
    stage('Build') { sh 'docker build -t chetna/app:v2 .' }
    stage('Push') { sh 'docker push chetna/app:v2' }
    stage('Deploy') { sh 'kubectl apply -f deployment.yaml' }
    stage('Verify') { sh 'kubectl rollout status deployment/chetna-deployment' }
  }
```

}

Flow: Git Push -> Jenkins Build -> Docker Push -> kubectl apply -> K8s Rolling Update

Result: New image auto-deployed, no manual kubectl.

Screenshot: Jenkins pipeline + kubectl rollout status SUCCESS

Exp 8 (5 marks): Record All Tools & Documents

Evidence in this report:

- Dockerfile, docker-compose.yml, deployment.yaml, service.yaml
- All commands from Exp 1-7
- Screenshots: docker ps, docker images, ping, kubectl get pods, rollout history
- Observations per experiment
- GitHub Repo: github.com/chetna-parmar/devops-lab2

Exp 9 (10 marks): Containerization & Orchestration Report

Architecture:

Dev -> GitHub -> Jenkins CI -> Docker Hub -> Minikube Cluster -> Deployment (2 Pods) -> Service (NodePort) -> User

Deployment Strategy: Rolling Update, maxSurge 1, maxUnavailable 1

Scaling Results: 2 -> 4 pods in 8 sec, CPU HPA 2-5 pods

Observations: Docker gives consistency, K8s gives self-healing, scaling, zero-downtime updates. If pod crashes, K8s auto-restarts.

Conclusion: Complete containerized workflow achieved by Chetna Parmar.